

learn that tools like Vim, sed, AWK, etc. are Turing complete. This knowledge about Turing completeness can be used to shed some light on exotic programming languages. Esoteric programming languages are tiny programming languages that are Turing complete. In layman's terms we can say that esoteric languages are powerful enough to do any task that is achievable by heavyweight languages like C, C++, Java, etc. Some of the joke languages and most of the non-English-based languages are also Turing complete.

The importance and relevance of exotic programming languages

Now we know that the languages we will be discussing here are Turing complete and are worthy of being called a 'programming language'. What about their relevance in the open source arena? In my opinion, these languages truly capture the ethos of open source program development. They neither have any commercial applications nor will they ever allow the developer to earn even a single rupee. No profit-seeking company will ever develop any of these languages. But each and every one of these languages has a small but fervent group of supporters responsible for development and maintenance. This underlines the fact that the real value of open source development is broadening the frontiers of knowledge and not just monetary benefit.

Another question that needs an answer is regarding the uses of exotic programming languages. If these languages are not widely used then why study or write about them? For starters, programming is not always about developing applications. Programming can also be used to test the skills of a potential candidate in an interview, to create puzzles that will baffle even the most accomplished programmer, for recreational purposes, etc. So, in essence, exotic programming languages are somewhat similar to crossword puzzles or word jumble games; they offer very good exercise for the mind. Over the years, thousands of pages have been filled about the features of different programming language paradigms like object oriented programming, structured programming, functional programming and logic programming. Also, computer magazines have published volumes about different languages like C, C++, Java, Python, etc. But none of them ever considered exotic programming languages seriously. So it will be pure fun for readers to get to know these weird languages.

Esoteric programming languages

Esoteric programming languages (also known as esolangs) are Turing complete languages in which program development is done with a tiny set of tokens. These languages are created to test the patience and perseverance of potential geeks. The sheer variety of languages available is quite amazing. In this section we will discuss just five esoteric programming languages. In fact, there are more than 750 entries in the wiki page about esoteric languages. We will discuss esolangs called Brainfuck, Ook!, Piet, Whitespace and Java2K.

Brainfuck

Brainfuck is the most widely used and popular esoteric language. But its awkward name often limits public discussions on it. The language is often called BF, Brainf*** or Brainf**k by the timid. So henceforth I will address the language as BF, which is known for its extreme minimalism. There are only eight commands and one instruction pointer in BF. A data pointer is used to point to the current cell being processed. The number of cells is theoretically infinite and all are initialised to zero. The eight commands are represented by the following eight symbols. They are:

- '<' (less than to increment data pointer)
- '>' (greater than to decrement data pointer)
- '+' (plus to increment the byte at the data pointer)
- '-' (minus to decrement the byte at the data pointer)
- '.' (dot to output the byte at the data pointer)
- ',' (comma to store one byte at the location pointed by the data pointer)
- '[' (opening square bracket to forward the instruction pointer to the next command after the closing square bracket if the byte at the data pointer is zero), and
- ']' (closing square bracket to move back the instruction pointer to the next command after the opening square bracket if the byte at the data pointer is non-zero).

Thus any BF program will consist of only these eight symbols.

Whitespaces are ignored by the BF interpreter. There are many open source interpreters available for processing BF. One of the most widely used interpreter and IDE for BF is an open source IDE called BFdev. The extensions for BF programs are *.b* and *.bf*. Writing programs in BF is considered a Herculean task. It is so difficult that if there is a God and if he ever lets me enter heaven, I will request him to write an operating system using BF – just as proof of his omniscience. Similarly, printing an arbitrarily large BF program might take up the whole magazine. So I will stick to a trivial task. The following is a BF program to print the text ‘*OPEN SOURCE FOR YOU*’ on the screen.

```

>+++++[>+++++++<.]>+++++++,
>+++++++[>+++++++<.]>,
>+++++++[>+++++++<.]>-,
>+++++++[>+++++++<.]>+++++++,
>+>[>+++++++<.]>+>,
>+++++++[>+++++++<.]>+>+,
>+++++++[>+++++++<.]>+++++++,
>+++++++[>+++++++<.]>+++++++,
>+++++++[>+++++++<.]>+>,
>+++++++[>+++++++<.]>+>+,
>+++++++[>+++++++<.]>+++++++,
>+++++++[>+++++++<.]>-,
>+>[>+++++++<.]>+>,
>+++++++[>+++++++<.]>,
>+++++++[>+++++++<.]>+++++++,
>+++++++[>+++++++<.]>+>,

```